

1.概要

上一回,说了如何采集视频(画面)并保存到文件,如果改成链接,就可以通过流媒体服务器(现有的),进行推流直播,不过缺少声音部分,接下来将采集声音并进行编码,然后完成同步,实现初版的视音频编码并保存文件(或推流).

先说一下解决方案:

采用 Qt 进行采集声音,通过定时器定时采集,然后通过转换器,转换为 ffmpeg 编码所需的 AVFrame 数据,数据为重采样的格式,然后进行编码,编码为 AAC,然后写入流.

简易的同步,就是写视频 AVpacket 和音频 AVPacket 的时候,会算出他们的 pts,谁的 pts 更小,就写谁的包到媒体流中.

2.方案的确定

为什么采集用 qt,而没有使用 ffmpeg 或者 sdl? 因为 qt 相对简单容易上手,使用定时器控制,流程简单,故障率低,而 ffmpeg 较难上手(需要按照名字打开设备,较繁琐),容易出现运行错误. SDL 也可以采集,也是很好的方案,因为之前写过使用 qt 采集的例子(视频会议),这里相当于在之前的版本上做出修改,与之前的关联性更强.

SDL 采集音频的代码如下

```
#include <SDL2/SDL.h>
#include <iostream>

const int SAMPLE_RATE = 44100;
const int CHANNELS = 2;
const int BUFFER_SIZE = 4096;

int main(int argc, char **argv) {
    // 初始化 SDL
    if (SDL_Init(SDL_INIT_AUDIO) != 0) {
        std::cerr << "无法初始化 SDL: " << SDL_GetError() << std::endl;
        return -1;
    }

    // 设置音频规格
    SDL_AudioSpec fmt;
    fmt.freq = SAMPLE_RATE;
    fmt.format = AUDIO_S16SYS;
    fmt.channels = CHANNELS;
    fmt.samples = BUFFER_SIZE;
    fmt.callback = NULL;

    // 打开音频设备并开始采集声音
    SDL_AudioDeviceID dev = SDL_OpenAudioDevice(NULL, 1, &fmt, NULL, 0);
    if (dev == 0) {
        std::cerr << "无法打开音频设备: " << SDL_GetError() << std::endl;
        return -1;
    }
    SDL_PauseAudioDevice(dev, 0);
```

```

// 循环采集并输出声音
while (true) {
    Uint8 buf[BUFFER_SIZE];
    SDL_QueueAudio(dev, buf, BUFFER_SIZE);
    // TODO: 处理采集到的声音数据
}

// 停止采集并关闭音频设备
SDL_CloseAudioDevice(dev);
SDL_Quit();

return 0;
}

```

其中，首先使用 `SDL_Init` 函数初始化 SDL，然后设置音频规格，并打开音频设备并开始采集声音。`SDL_OpenAudioDevice` 函数返回的 `dev` 是音频设备的 ID，可以用于后续的操作。

在循环中，使用 `SDL_QueueAudio` 函数采集声音数据，并将其存储在缓冲区中。`buf` 是存储声音数据的缓冲区，`BUFFER_SIZE` 是缓冲区的大小，可以根据需要进行调整。

最后，在程序结束时，使用 `SDL_CloseAudioDevice` 函数关闭音频设备，并使用 `SDL_Quit` 函数结束 SDL 的使用。上面采集的音频数据是 PCM 格式数据，那么就需要

如果有想法可以考虑使用上面的代码利用 SDL 采集声音，这里将介绍使用 qt `QAudioInput` 采集。

3.音频采集

首先,使用 `QAudioInput` 需要添加 Qt 的多媒体库

需要在 pro 文件中添加 `QT+= multimedia`

采集的类

```

#ifndef AUDIO_READ_H
#define AUDIO_READ_H

```

```

#include <QObject>
#include "common.h"
#include<QTimer>
#include<vector>
#include<QMessageBox>
#include<QTime>
#include<QAudioInput>
#include<QAudioFormat>

```

```

#define AVCODEC_MAX_AUDIO_FRAME_SIZE 192000///// 1 second of 48khz 32bit audio

```

//计算方法 采样率 * 采样时间间隔 * 一次采样字节数 = 数据量

//单通道 16 位 一次采样 2 字节 双通道 16 位 一次采样 4 字节

//aac 数据是一帧 1024 采样点 ffmpeg 使用 fltp 是 float 一个点 双声道*2 就是*8 即 8192 字节

//PCM 采样 可以是 48kHz 或者 44.1kHz，以 1024 点为例，换算

//48kHz : 1000 = 1024 : x x 为 21.3 我们去采样时间间隔为 20ms

```

//单声道 队列投递一次字节数
#define OneAudioSize (4096)
//采样频率
#define AudioCollectFrequency (44100)
//采样间隔
#define AUDIO_INTERVAL (20)
//定时不准 一般延迟一些
//读一帧可能会有噪音 读两帧试试
//一次读取 8 帧 32768 4096
#define FrameOnce (1)
//声道数
#define AudioChannelCount (2)

class Audio_Read : public QObject
{
    Q_OBJECT
signals:
    void SIG_sendAudioFrameData( uint8_t* buf, int buffer_size);
public:
    Audio_Read();
    ~Audio_Read();

public slots:
    void slot_readMore(); //定时获取音频数据投递到队列
    void slot_closeAudio(); //暂停采集
    void slot_openAudio(); //开始采集
    void UnInit(); //回收

private:
    QAudioInput* audio_in; //Qt 音频采集对象
    QIODevice *myBuffer_in; //IO 设备，读写数据
    QAudioFormat format; //音频设备格式

    //采集状态
    enum ENUM_AUDIO_STATE{state_stop , state_play , state_pause };
    int m_playState;

    QTimer *timer; //定时器
    SwrContext *swr; // 用于音频的转换
    int nb_samples;
    std::vector<uint8_t> m_audiobuff; //缓冲区排队执行
    int m_buffPos; // 未取走数据的起始位置
};

#endif // AUDIO_READ_H
#include "audio_read.h"

```

```

Audio_Read::Audio_Read()
{
    //声卡采样格式
    // set up the format you want, eg.
    format.setSampleRate(AudioCollectFrequency);
    format.setChannelCount(2);
    format.setSampleSize(16);
    format.setCodec("audio/pcm");
    format.setByteOrder(QAudioFormat::LittleEndian);
    //format.setByteOrder(QAudioFormat::BigEndian);
    format.setSampleType(QAudioFormat::UnsignedInt);
    //format.setSampleType(QAudioFormat::SignedInt);
    QAudioDeviceInfo info = QAudioDeviceInfo::defaultInputDevice();
    if (!info.isFormatSupported(format)) {
        // qWarning() << "default format not supported try to use nearest";
        QMessageBox::information(NULL, "提示", "打开音频设备失败");
        format = info.nearestFormat(format);
    }

    audio_in = NULL;
    m_playState = state_stop;

    timer = new QTimer;

    connect( timer , SIGNAL(timeout())
            , this , SLOT(slot_readMore()) );

    m_buffPos = 0;
}

Audio_Read::~~Audio_Read()
{
    this->UnInit();
    delete audio_in;
}

//关于数据点个数 aac 采样点 一帧 1024 频率 44.1kHz 那么采样间隔就是 1024/44.1= 23.2 ms
//一次读取两帧 aac 即 2048 个采样点
void Audio_Read::slot_readMore()
{
    //    ///视频没有开始采集 不采集音频

    if (!audio_in)
        return;

    std::vector<uint8_t> multipacketbuffer;

    qint64 len = audio_in->bytesReady();

```

```

if (len < OneAudioSize )
{
    return;
}
//一次性都读出来
if( m_buffPos == 0)
{
    m_audiobuff.resize( len );
}else
{
    int nSize = m_audiobuff.size();
    uint8_t* pBegin = &*m_audiobuff.begin();

    std::vector<uint8_t> tempbuff(nSize - m_buffPos , 0 );
    uint8_t* pTemp = &*tempbuff.begin();
    //先保存到临时空间 , 然后申请新空间
    memcpy( pTemp , pBegin + m_buffPos , nSize - m_buffPos);

    m_audiobuff.resize( len + tempbuff.size() );
    pBegin = &*m_audiobuff.begin();

    memcpy( pBegin , pTemp , tempbuff.size() );
    m_buffPos = tempbuff.size();
}

uint8_t* pAudioBuff = &*m_audiobuff.begin() + m_buffPos;

int offset = 0;
int packSize = len;

//数组首地址
// uint8_t * pInData = (uint8_t*)&*m_buffer.begin() ;

//这次不读走 后面就丢了
while( packSize ){
    qint64 l = myBuffer_in->read( (char*)(pAudioBuff + offset) , packSize );
    if( l > 0 )
    {
        packSize -= l;
        offset += l;
    }
}

//积累了多少帧
int nFrameCount = ( m_audiobuff.size()/OneAudioSize );
// qDebug() << "nFrameCount = " << nFrameCount;

multipacketbuffer.resize( OneAudioSize*nFrameCount );

```

```
//所有帧数组首地址
uint8_t * in_buffer = (uint8_t*)&*multipacketbuffer.begin() ;

memcpy( in_buffer , &*m_audiobuff.begin() , OneAudioSize*nFrameCount );
m_buffPos = OneAudioSize*nFrameCount;
if( m_buffPos == m_audiobuff.size() )
    m_buffPos = 0;

// nb_samples 和 frame_size = 1024
// nb_samples 采样数
//一帧数据量：1024*2*av_get_bytes_per_sample(s16) = 4096 个字节。
//会编码：88200/(1024*2*av_get_bytes_per_sample(s16)) = 21.5 帧数据
// aac 规定每帧音频只能有 1024 个采样

//采集的 pcm 数据是 s16 模式 是 uint 而编码的时候，使用的是 fltp 需要转换 使用
swr_convert

//1024 字节 元素 float *4
uint8_t * out_buffer[2]; //左右声道数据
out_buffer[0] = (uint8_t *)malloc( OneAudioSize*nFrameCount );
memset( out_buffer[0] , 0 ,OneAudioSize*nFrameCount );

out_buffer[1] = (uint8_t *)malloc( OneAudioSize*nFrameCount );
memset( out_buffer[1] , 0 ,OneAudioSize*nFrameCount );

// 关于长度 采样数 即 1024
int count=swr_convert(swr, out_buffer , nFrameCount*OneAudioSize/4,
                    (const uint8_t
**)&*pInData*/in_buffer,nFrameCount*OneAudioSize/4); //len 为 4096

//每次 读取 一帧
for( int n = 0 ; n < nFrameCount ; n++)
{
    uint8_t * audio_left = out_buffer[0] + n* OneAudioSize ;
    uint8_t * audio_right = out_buffer[1] + n* OneAudioSize ;

    //pData 存储 PCM 数据 长度 4096 小端
    //转换为 ffmpeg FLTP 平面模式 LLLLLLLLLLLLLLLLLRRRRRRRRRRRRRRRRR

    uint8_t * buffer = (uint8_t *)malloc( OneAudioSize * AudioChannelCount );
    memset( buffer , 0 ,OneAudioSize * AudioChannelCount );

    //第一帧左声道
    memcpy( buffer , audio_left , OneAudioSize );
    //第一帧右声道
    memcpy( buffer + OneAudioSize , audio_right , OneAudioSize );

    Q_EMIT SIG_sendAudioFrameData( (uint8_t*)buffer, OneAudioSize *
AudioChannelCount );
```

```

    }

    delete [] out_buffer[0];
    delete [] out_buffer[1];
}

void Audio_Read::UnInit()
{
    if(timer)
        timer->stop();
    if(audio_in)
        audio_in->stop();
}

void Audio_Read::slot_openAudio()
{
    if( m_playState == state_stop )
    {
        audio_in = new QAudioInput(format, this);
        myBuffer_in = audio_in->start();
    }
    else
    {
        if(audio_in)
        {
            delete audio_in;
            audio_in = new QAudioInput(format, this);
            myBuffer_in = audio_in->start();
        }
    }
    m_playState = state_play;

    //重采样设置选项-----
    enum AVSampleFormat in_sample_fmt; //输入的采样格式
    enum AVSampleFormat out_sample_fmt; //输出的采样格式 16bit PCM
    int in_sample_rate; //输入的采样率
    int out_sample_rate; //输出的采样率
    int in_ch_layout; //输入的声道布局
    int out_ch_layout; //输出的声道布局

    //输入的采样格式
    in_sample_fmt = AV_SAMPLE_FMT_S16;
    //输出的采样格式 32bit PCM
    out_sample_fmt = AV_SAMPLE_FMT_FLTP;

    //输入的采样率

```

```

    in_sample_rate = AudioCollectFrequency;
    //输出的采样率
    out_sample_rate = 44100/*48000*/;

    //输入的声道布局
    in_ch_layout = AV_CH_LAYOUT_STEREO;
    //输出的声道布局
    out_ch_layout = AV_CH_LAYOUT_STEREO;

    swr = swr_alloc_set_opts(nullptr, out_ch_layout, out_sample_fmt, out_sample_rate,
                             in_ch_layout, in_sample_fmt, in_sample_rate,
0, nullptr);

    swr_init(swr);

    timer->start( AUDIO_INTERVAL );
}

void Audio_Read::slot_closeAudio()
{
    if(timer) {
        timer->stop();
    }
    if( m_playState == state_play )
    {
        m_playState = state_pause;
        if(audio_in)
            audio_in->stop();
    }
}

```

采集到的数据最终会以信号 SIG_sendAudioFrameData(uint8_t*)buffer, OneAudioSize * AudioChannelCount)的形式发出去,
写文件类接收并使用槽函数处理即可.

4.编码流程

4.1 添加流

```

void SaveVideoFileThread::add_audio_stream(OutputStream *ost, AVFormatContext
*oc, AVCodec **codec, enum AVCodecID codec_id )
{
    AVCodecContext *c;
    int i;

    /* find the encoder */
    *codec = avcodec_find_encoder(codec_id);
}

```



```

if (!(*codec)) {
    fprintf(stderr, "Could not find encoder for '%s'\n",
               avcodec_get_name(codec_id));
    exit(1);
}

ost->st = avformat_new_stream(oc, NULL);
if (!ost->st) {
    fprintf(stderr, "Could not allocate stream\n");
    exit(1);
}
ost->st->id = oc->nb_streams-1;
c = avcodec_alloc_context3(*codec);
if (!c) {
    fprintf(stderr, "Could not alloc an encoding context\n");
    exit(1);
}
ost->enc = c;

c->sample_fmt = AV_SAMPLE_FMT_FLTP;
c->bit_rate = 64000;
c->sample_rate = 44100;

c->channels = av_get_channel_layout_nb_channels(c->channel_layout);
c->channel_layout = AV_CH_LAYOUT_STEREO;

ost->st->time_base = { 1, c->sample_rate };

/* Some formats want stream headers to be separate. */
if (oc->oformat->flags & AVFMT_GLOBALHEADER)
    c->flags |= AV_CODEC_FLAG_GLOBAL_HEADER;
}

```

4.2 打开音频

```

void SaveVideoFileThread::open_audio(AVFormatContext *oc, AVCodec *codec, OutputStream
*ost )
{
    AVCodecContext *c;
    int nb_samples;
    int ret;

    c = ost->enc;

    /* open it */
    // av_dict_copy(&opt, opt_arg, 0);
    ret = avcodec_open2(c, codec, NULL );
}

```

```

if (ret < 0) {
    fprintf(stderr, "Could not open audio codec \n" );
    exit(1);
}

nb_samples = c->frame_size;

ost->frame = av_frame_alloc();

if (!ost->frame) {
    fprintf(stderr, "Error allocating an audio frame\n");
    exit(1);
}

ost->frame->format = c->sample_fmt;
ost->frame->channel_layout = c->channel_layout;
ost->frame->sample_rate = c->sample_rate;
ost->frame->nb_samples = nb_samples;

//申请空间
mAudioOneFrameSize = av_samples_get_buffer_size(NULL, c->channels,
c->frame_size, c->sample_fmt, 1);
qDebug() << "mAudioOneFrameSize:" << mAudioOneFrameSize;

ost->frameBuffer = (uint8_t *)av_malloc(mAudioOneFrameSize);
ost->frameBufferSize = mAudioOneFrameSize;

///这句话必须要(设置这个 frame 里面的采样点个数)
int oneChannelBufferSize = mAudioOneFrameSize / c->channels; //计算出一个声道的
数据
int nb_samplesize = oneChannelBufferSize /
av_get_bytes_per_sample(c->sample_fmt); //计算出采样点个数
ost->frame->nb_samples = nb_samplesize;

///这 2 种方式都可以
// avcodec_fill_audio_frame(ost->frame, c->channels, c->sample_fmt, (const
uint8_t*)ost->frameBuffer, mAudioOneFrameSize, 0);
av_samples_fill_arrays(ost->frame->data, ost->frame->linesize,
ost->frameBuffer, c->channels, ost->frame->nb_samples, c->sample_fmt, 0);

/* copy the stream parameters to the muxer */
ret = avcodec_parameters_from_context(ost->st->codecpars, c);
if (ret < 0) {
    fprintf(stderr, "Could not copy the stream parameters\n");
    exit(1);
}
}

```

4.3 写音频帧

```
void SaveVideoFileThread::slot_writeAudioFrameData(uint8_t *picture_buf, int
buffer_size)
{
    AVCodecContext *c;
    c = audio_st.enc;

    memcpy( audio_st.frameBuffer , picture_buf , buffer_size );

    free( picture_buf );

    audio_st.frame->pts = audio_st.next_pts;
    audio_st.next_pts += audio_st.frame->nb_samples;

    AVRational rational;
    rational.num = 1;
    rational.den = c->sample_rate;
    audio_st.frame->pts = av_rescale_q(audio_st.samples_count, rational, c->time_base);
    audio_st.samples_count += audio_st.frame->nb_samples;

    write_frame(oc, audio_st.enc, audio_st.st, audio_st.frame);
}
```

4.4 关闭

关闭回收的代码是一样的，并没有变化